Extract Insights from Visual Data on Azure

Vision, image & video generation, content understanding, and search

Rick Beerendonk
rick@oblicum.com

glasspaper oblicum

What You'll Learn

- Send an image to a model and ask questions about it
- Generate images and video, and edit part of an image
- Pull named fields out of forms, invoices, and mixed media
- Choose between a model, an analyzer, and a document model
- Turn a pile of files into something searchable

Problem: 90% of Company Content Is Not a Database Row

Scanned invoices. Photos from the field. Recorded meetings. Slide decks in a share.

- ❌ You cannot query a PDF
- ❌ You cannot filter a photo
- ❌ Someone retypes the invoice total into the finance system by hand

Need: turn pixels and pages into fields, text, and searchable records.

This presentation is four different tools for that one job — and knowing which to reach for is most of the skill.

glasspaper oblicum

Which Tool for Which Job?

```
What do you have, and what do you want back?

  "Describe / reason about this image"          ──▶ multimodal model  (gpt-4.1)
       open-ended answer, no fixed schema

  "Give me these exact fields"                  ──▶ Document Intelligence
       invoice total, IBAN, signature box           forms and documents

  "Same, but the input might be audio,          ──▶ Content Understanding
  video, or an image too"                           one analyzer, many media types

  "Make me a picture / a video"                 ──▶ gpt-image-1 / FLUX / Sora

  "Let people search 100,000 of these"          ──▶ AI Search + skillset
```

The dividing line: a *model* gives you an answer; an *analyzer* gives you the same fields every time.

glasspaper oblicum

Everything Big Is Asynchronous

Image analysis, video generation, and document analysis all take longer than one HTTP request.

```
1  you submit the job          begin_analyze(...) / videos.create(...)
       |
       ▼                        you get back an operation id, not a result
2  service works in the background
       |
3  you poll                    "is it done yet?"   status: running
       |                       "is it done yet?"   status: running
       |                       "is it done yet?"   status: succeeded
       ▼
4  you fetch the result        poller.result()  /  download the file
```

That is why the SDK calls start with `begin_` and return a *poller*. When you see `Operation-Location` in a response header, you are in step 3.

glasspaper oblicum

Vision-Enabled Generative AI

glasspaper

oblicum

Multimodal Models

Model	Provider	Notes
`gpt-4.1`	OpenAI	Supports Responses API and ChatCompletions
`gpt-4.1-mini`	OpenAI	Lighter, cost-effective variant
`Phi-4-multimodal-instruct`	Microsoft	ChatCompletions API only

Image input formats: web URL or Base64 data URL: `{data}`

glasspaper oblicum

Responses API (gpt-4.1, gpt-4.1-mini)

[EXAM]

```python
from openai import AzureOpenAI
import base64

client = AzureOpenAI(azure_endpoint=ENDPOINT, api_key=KEY, api_version="2025-01-01-preview")

with open("image.jpg", "rb") as f:
    b64 = base64.b64encode(f.read()).decode("utf-8")

response = client.responses.create(
    model=DEPLOYMENT,
    input=[
        {
            "role": "developer",
            "content": [
                {"type": "input_text", "text": "What is in this image?"},
                {"type": "input_image", "image_url": f"{b64}"},
            ],
        }
    ],
)
print(response.output_text)
```

Note: role is `"developer"`; image type is `"input_image"`

ChatCompletions API (gpt-4.1, Phi-4)

[EXAM]

```
response = client.chat.completions.create(
    model=DEPLOYMENT,
    messages=[
        {"role": "system", "content": "You analyze images."},
        {
            "role": "user",
            "content": [
                {"type": "text", "text": "What is in this image?"},
                {"type": "image_url",
                 "image_url": {"url": f"{b64}"}},
            ],
        },
    ],
)
print(response.choices[0].message.content)
```

Note: role is `"system"`; image type is `"image_url"` (nested `{"url": ...}`)

Multimodal request shape: the image belongs in the **user message content array** as an `image_url` item, alongside the `text` item. Base64-encoding it into the

Image Generation

glasspaper

oblicum

Retrieval Practice — Lab 1

Pause here and complete Lab 1 before continuing.

glasspaper oblicum

Image Generation — gpt-image-1 and FLUX

[EXAM]

Models:

```python
import base64
from openai import AzureOpenAI

client = AzureOpenAI(azure_endpoint=ENDPOINT, api_key=KEY, api_version=API_VERSION)

img_results = client.images.generate(
    model=MODEL_DEPLOYMENT,
    prompt="A golden retriever running through autumn leaves, photorealistic",
    n=1,
    size="1024x1024",
)

image_bytes = base64.b64decode(img_results.data[0].b64_json)
with open("output.png", "wb") as f:
    f.write(image_bytes)
```

Response: `data[0].b64_json` — always returned as base64; decode to get raw bytes

Keeping a Real Product Recognisable

[EXAM]

Generating a marketing scene around a real product usually distorts it — the label, shape, or colour drifts. Raise `input_fidelity` to preserve the reference:

```
result = client.images.edit(
    model="gpt-image-1",
    image=open("product.png", "rb"),
    prompt="Place the bottle on a marble kitchen counter, morning light",
    input_fidelity="high",     # keep the product identity
)
```

Setting	Effect
`input_fidelity="high"`	Faces and distinctive product details stay close to the input
Default fidelity	More creative freedom, weaker identity preservation

Not a substitute: a groundedness filter checks text claims, prompt wording alone does not lock appearance, and temperature controls variability rather than fidelity.

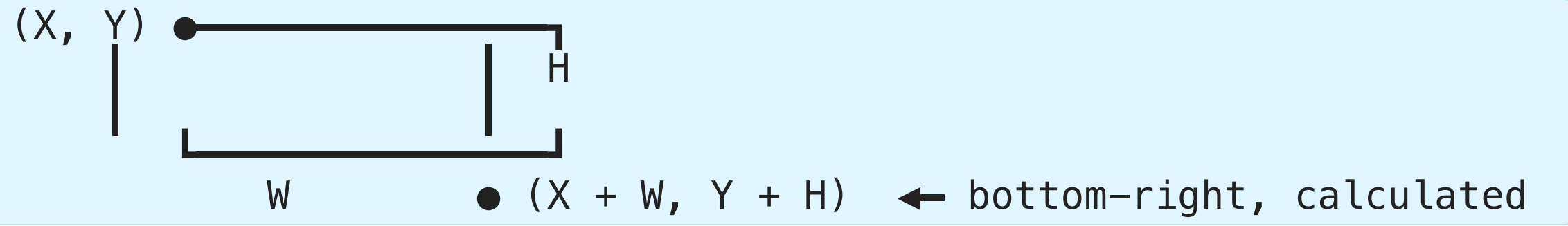
Detecting Brands and Reading Bounding Boxes

[EXAM]

Azure Vision returns a `brands` collection with a confidence score and a rectangle per detection:

```
foreach (var brand in analysis.Brands)
{
    if (brand.Confidence >= 0.75)
        Console.WriteLine($"{brand.Name} at {brand.Rectangle.X},{brand.Rectangle.Y}");
}
```

The rectangle gives the top-left corner plus a size — not two corners:



Property	Meaning
`X`, `Y`	**Top-left** corner coordinates

Trap: `X` and `Y` are never the bottom-right corner.

glasspaper obliqUM

Image Editing — Inpainting with a Mask

[EXAM]

Use case: an existing image contains something to remove (competitor logo in the background, unwanted object) — edit only that region instead of regenerating the whole image.

API: `client.images.edit()` (not `generate`) — supplies the source image plus a **mask PNG** where transparent pixels mark the region to be re-painted.

```
result = client.images.edit(
    model="gpt-image-1",
    image=open("product-photo.png", "rb"),
    mask=open("mask.png", "rb"),       # transparent region = paint here; opaque = keep
    prompt="Replace the logo with a plain wooden wall, matching the existing lighting",
    size="1024x1024",
)
image_bytes = base64.b64decode(result.data[0].b64_json)
```

Mask rules: same dimensions as source; PNG with alpha channel; only transparent pixels are repainted — the rest stays pixel-identical.

Not the same as `generate`: `generate` makes a new image; `edit` preserves the original everywhere except the masked area.

Retrieval Practice — Lab 2

Pause here and complete Lab 2, then check its solution.

glasspaper oblicum

Video Generation

glasspaper oblicum

Sora Video Generation

[EXAM]

Models: `sora-2` (standard), `sora-2-pro`

Sizes: `"1280x720"` (landscape) or `"720x1280"` (portrait)
Durations: `"4"`, `"8"`, or `"12"` (seconds, as string value)
Concurrency limit: max 2 jobs at a time

Pattern: Create → Poll → Download

```
video = client.videos.create(
    model="sora-2",
    prompt="Ocean waves at sunrise, cinematic slow motion",
    size="1280x720",
    seconds="8",
)

# Poll until done
while video.status not in ["completed", "failed", "cancelled"]:
    video = client.videos.retrieve(video.id)

# Download
```

Sora — Remix and Reference Image

Video remix (modify an existing video):

```
remixed = client.videos.remix(
    video_id=original_video.id,
    prompt="Change the setting to winter, keep the same scene composition",
)
```

Reference image (constrain the generated scene):

```
video = client.videos.create(
    model="sora-2",
    prompt="A cat walks through this landscape",
    size="1280x720",
    seconds="8",
    input_reference=open("landscape.png", "rb"),
)
```

Constraint: Reference images must not contain human faces

glasspaper oblicum

Editing Video Without Regenerating It

[EXAM]

A finished clip contains an unwanted watermark or object. Regenerating produces a **different video** — instead, apply a **mask-based inpainting edit** to that region, exactly as with images.

Approach	Result
Mask-based inpainting	Removes the artifact, keeps the rest of the footage intact
Change the prompt and re-run	New generation, so the approved scene is lost
Crop the frame	Removes wanted content and changes the aspect ratio
Adjust the guidance scale	Changes generation behavior, not an existing artifact

Rule: to change *part* of an existing asset, edit with a mask. To change *the whole* asset, generate again.

Azure Content Understanding

glasspaper

ContentUnderstandingClient

[EXAM]

Package: `azure-ai-contentunderstanding`; **API version:** `2025-11-01`

Prerequisite: Foundry resource must have these deployed:

- `GPT-4.1` or `GPT-4.1-mini`
- `text-embedding-3-large`

```
from azure.ai.contentunderstanding import ContentUnderstandingClient
from azure.ai.contentunderstanding.models import AnalysisInput
from azure.core.credentials import AzureKeyCredential

client = ContentUnderstandingClient(
    endpoint=ENDPOINT,
    credential=AzureKeyCredential(KEY),
    api_version="2025-11-01"
)

poller = client.begin_analyze(
    analyzer_id="prebuilt-invoice",
    inputs=[AnalysisInput(data=open("invoice.pdf", "rb").read())],
```

Prebuilt Analyzers

[EXAM]

Analyzer	Purpose
`prebuilt-image`	General image analysis
`prebuilt-receipt`	Merchant, items, totals
`prebuilt-invoice`	Vendor, line items, tax, totals

Supported media: Images, documents, audio, video

Choosing between the document analyzers:

Requirement	Analyzer
Tables, QR codes, and where things sit on a page	`prebuilt-layout`
Search-ready semantic Markdown for a	`prebuilt`

Single-File and Multi-File Tasks

[EXAM]

Task type decides how many documents the analyzer can reason over at once.

Task	Mode	Reasoning
Single-file	**standard**	One document analyzed in isolation — high volume, low cost
Multi-file	**pro**	Several documents **plus reference data**, compared together

```
   Single-file / standard      each invoice → fields

 Multi-file / pro             invoice + purchase order
                                 + vendor contracts (reference data)
                              → validated, reconciled result
```

Use pro mode when the requirement is cross-document: matching an invoice to its purchase order, or validating it against contract terms. Standard mode cannot compare documents, so it cannot answer “does this invoice agree with the PO?”

Both can run in one solution: bulk extraction in single-file standard, exception validation in multi-file pro.

glasspaper oblicum

Field Schema and Confidence Thresholds

[EXAM]

Field extraction methods:

Method	Description	Example
`extract`	Pull value as-is	`InvoiceTotal: "€1,245.00"`

Confidence thresholds:

Score range	Action
> 0.9	Automate (no review needed)

Field properties: `valueString`, `confidence`, `source` (bounding polygon for grounding)

For scanned troubleshooting documents, enable `estimateFieldSourceAndConfidence` to return per-field confidence and source location for review.

Generated descriptions — for example a color scheme for a video segment — need field type **`string`** with method **`generate`**. `extract` and `classify` read or label existing content, and a non-string type cannot hold the generated text.

Custom Analyzers

[EXAM]

Create via SDK:

```
poller = client.begin_create_analyzer(
    "expense-report-analyzer",
    body={
        "description": "Extracts expense report totals and categories",
        "baseAnalyzerId": "prebuilt-invoice",
        "fieldSchema": {
            "fields": {
                "ExpenseCategory": {"type": "string", "method": "classify",
                                    "enum": ["Travel", "Meals", "Software", "Other"]},
                "TotalAmount": {"type": "number", "method": "extract"},
                "Justification": {"type": "string", "method": "generate"},
            }
        },
        "models": {"completion": {"modelId": "gpt-4.1"},
                   "embedding": {"modelId": "text-embedding-3-large"}},
    }
)
```

Create via REST: `PUT {endpoint}/contentunderstanding/analyzers/{name}?api-version=2025-11-01`

Content Understanding: Build vs. Consume

Two complementary workflows:

Workflow	Purpose	Typical tool
Build an analyzer	Define fields and extraction/generation methods	Content Understanding Studio, SDK, or REST
Consume an analyzer	Submit files and process structured results	Python SDK or REST client application

Prerequisites:

- A Microsoft Foundry resource or project
- The resource endpoint and an API key, or Microsoft Entra ID authentication
- `GPT-4.1` or `GPT-4.1-mini` completion deployment
- `text-embedding-3-large` embedding deployment
- Python 3.9+ for the Python SDK

Studio is useful for: visually creating schemas, testing sample content, and inspecting extracted fields before moving the definition into code.

glasspaper oblicum

Content Understanding Client Workflow

```python
from azure.ai.contentunderstanding import ContentUnderstandingClient
from azure.ai.contentunderstanding.models import AnalysisInput
from azure.core.credentials import AzureKeyCredential

client = ContentUnderstandingClient(
    endpoint=ENDPOINT,
    credential=AzureKeyCredential(KEY),
    api_version="2025-11-01",
)

poller = client.begin_analyze(
    analyzer_id="expense-report-analyzer",
    inputs=[AnalysisInput(data=open("expense.pdf", "rb").read())],
)
result = poller.result()

for content in result.contents:
    print(content.markdown)
    print(content.fields)
```

REST alternative: submit the file to the analyzer endpoint, poll the returned operation URL, then process the JSON result. Use REST when the client language

When the Answer Is Content Understanding

[EXAM]

Reach for an analyzer when the requirement combines **structure**, **more than one modality**, and **no model training**.

Requirement in the scenario	Why Content Understanding
Preserve structure across mixed-format documents	An analyzer returns a schema, not free-form prose
Multipage tables **without training a model**	Prebuilt reasoning, no labeled dataset needed
Judge visual **and** textual evidence together	Multimodal analysis in one pass
Page citations with bounding polygons	Grounding metadata is returned with the fields
Named business fields from **varied layouts**	A custom analyzer generalizes beyond one template

Rejected alternative	Why
Azure Language text analysis	Text only — no layout, no tables
Chat completions or multimodal Responses	Free-form output, not structure-aware extraction
An Azure Machine Learning model	Requires building and training your own model
Document Layout / Document Extraction	Layout only, or no multimodal citation metadata
GenAI Prompt	Generates content instead of extracting it
`prebuilt-layout`	Returns structure, but not named business fields

Azure Document Intelligence

glasspaper

Document Intelligence Models

[EXAM]

Document Analysis:

Model	Extracts
`read`	Text + language detection (printed + handwritten)
`layout`	Text + tables + selection marks + key-value pairs

Prebuilt: Invoice, Receipt, Bank statement, W-2, 1040, ID document, Health insurance card, and more

Custom:

Type	Use Case
Custom Template	Fixed-layout forms; supports 100+ languages
Custom Neural	Variable-layout documents
Composed	Multiple custom models auto-classify and route

Input limits: < 500 MB, 50×50 to 10,000×10,000 px; PDF ≤ 17×17 in; no password-protected files

Retrieval Practice — Lab 3

Pause here and complete Lab 3, then check its solution.

glasspaper oblicum

DocumentAnalysisClient

```python
from azure.ai.formrecognizer import DocumentAnalysisClient
from azure.core.credentials import AzureKeyCredential

client = DocumentAnalysisClient(
    endpoint=ENDPOINT,
    credential=AzureKeyCredential(KEY)
)

poller = client.begin_analyze_document_from_url(
    model_id="prebuilt-invoice",
    document_url="https://example.com/invoice.pdf"
)
result = poller.result()

for doc in result.documents:
    for name, field in doc.fields.items():
        print(f"{name}: {field.content} (confidence: {field.confidence:.2f})")
```

Document: `begin_analyze_document(model_id, document=bytes)` **for binary input**

Studio: `documentintelligence.ai.azure.com` — label training data; test prebuilt models

Layout Model: Markdown Output and Page Chunks

[EXAM]

Use case: Extract structured text preserving layout — for RAG chunking, search indexing

SDK (v4): `azure-ai-documentintelligence` → `DocumentIntelligenceClient`

```python
from azure.ai.documentintelligence import DocumentIntelligenceClient
from azure.ai.documentintelligence.models import (
    AnalyzeDocumentRequest, ContentFormat
)
from azure.core.credentials import AzureKeyCredential

client = DocumentIntelligenceClient(ENDPOINT, AzureKeyCredential(KEY))

# Analyze pages 1–3 only; output as Markdown
poller = client.begin_analyze_document(
    "prebuilt-layout",
    AnalyzeDocumentRequest(url_source="https://example.com/report.pdf"),
    pages="1-3",
    output_content_format=ContentFormat.MARKDOWN,
)
result = poller.result()
```

Retrieval Practice — Lab 4

Pause here and complete Lab 4, then check its solution.

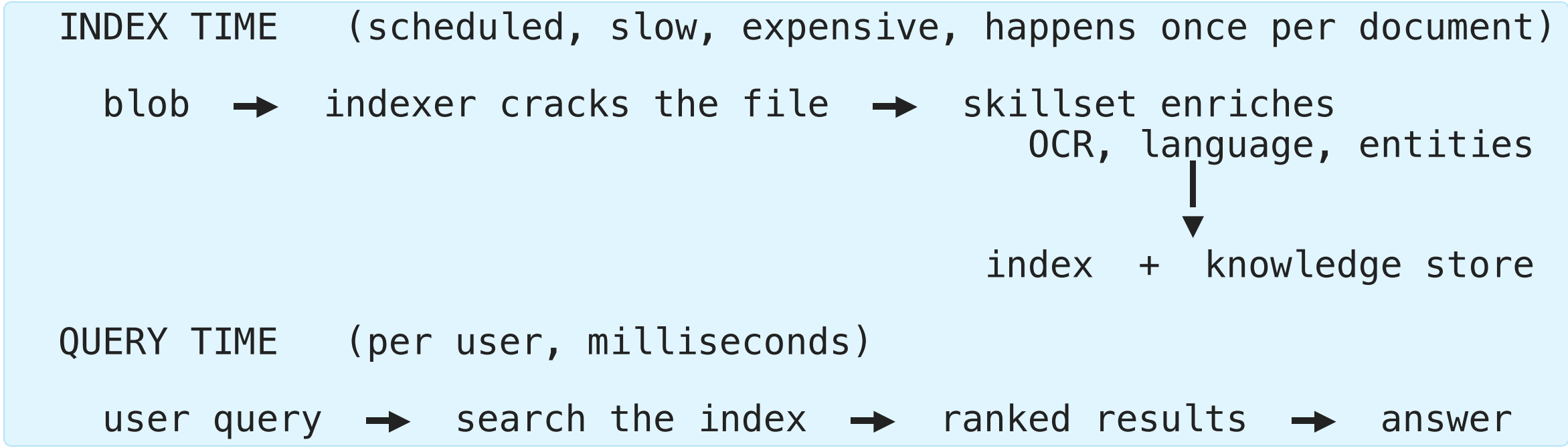
glasspaper oblicum

Azure AI Search

glasspaper

Two Timelines, Not One

The most common confusion with AI Search: enrichment does **not** happen when a user searches.



Consequence: adding a skill changes nothing until you re-run the indexer. Everything a user can search for had to be computed at index time.

glasspaper oblicum

Enrichment Pipeline

[EXAM]

```
Data Source → Indexer → Skillset → Index ⟶ Knowledge Store
                                        ↓
                                     Search
```

Components:

Component	Purpose
Data source	Blob Storage, SQL Database, Cosmos DB, SharePoint
	Cracks documents; orchestrates enrichment; runs on schedule

Enrichment tracks produced by indexer:

- `metadata` (file name, URL, type)
- `content` (extracted text)
- `normalized_images` (decoded images for OCR/captioning)
- `language`, `merged_content` (text + OCR combined)

For embedded PDF images that must be OCR-ready while retaining source citations, extract them into the indexer's `normalized_images` collection.

Built-in AI Skills (Foundry Tools)

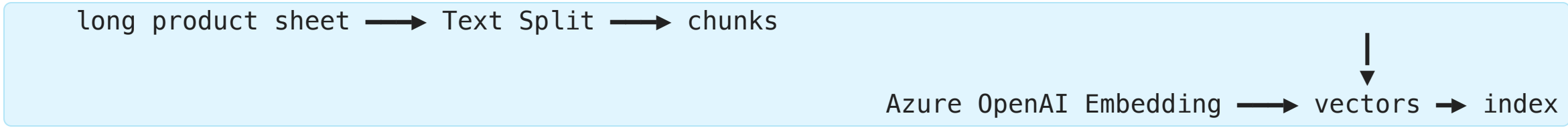
[EXAM]

Requires: Azure AI Foundry Tools (multi-service) resource in the same region as AI Search

Skill	Output
Language detection	`languageCode`

Free tier limit: ≤ 20 documents per indexer run with AI enrichment

For semantic and vector retrieval you need exactly two skills:



Entity extraction, language detection, and key phrase extraction enrich text but produce **no vectors**; Merge combines fields rather than chunking.

For scanned invoices, the skill that makes the text searchable is OCR. Text Split chunks text that already exists, Translation converts language, and Image Analysis describes visual content instead of extracting the invoice text.

Build a Knowledge Mining Solution

Typical Azure AI Search workflow:

1. Store source documents in Azure Blob Storage
2. Create a data source that connects the storage account
3. Define an index with searchable, filterable, and vector fields
4. Add a skillset for OCR, language, entities, key phrases, PII, or captions
5. Create an indexer to crack, enrich, and load the documents
6. Query with keyword, semantic, vector, or hybrid search

Hybrid search is usually the best default for RAG: lexical matching finds exact terms while vector search captures semantic similarity.

Grounding loop: search → select relevant chunks → provide citations and context → generate a response → evaluate groundedness.

Choosing a Retrieval Mode

[EXAM]

Requirement	Mode
Search **private** product sheets at all	**Azure AI Search**
Match a question to differently worded source text	**Vector search**
Exact codes **and** natural-language descriptions	**Hybrid search**
Reorder results that were already retrieved	Semantic ranking
Autocomplete while the user types	Suggesters
Tokenize and normalize text for lexical matching	Analyzers

Traps: semantic ranking only reranks what retrieval already returned, keyword-only search misses paraphrases, and vector-only search weakens exact code matching. Bing queries the public web, Translator converts languages, and Document Intelligence extracts content — none of them provide a private search index.

Knowledge Store

[EXAM]

Persists enriched content outside the search index — for downstream analytics or export

Projection types:

Type	Use Case

Why knowledge store? Enriched data survives index rebuilds; enables custom analytics pipelines; images can be exported separately from text content

Choosing a projection: unstructured enriched JSON → **object** projection; scanned text destined for rows and analytics → **table** projection. Reversing them stores tabular rows as loose JSON objects and forces unstructured content into a relational shape it does not fit.

Search Security Operations

[EXAM]

Document-level filtering — three steps, all required:

1. Add the **allowed groups** to every indexed document as a filterable field
2. Retrieve the signed-in user's **group memberships**
3. Pass those groups as a **filter** on each search request

```
filter=groups/any(g: search.in(g, 'sales,finance'))
```

One index per group does not scale, an Entra access token authenticates but does not filter documents, and fetching *all* groups returns memberships that are not the user's.

Rotating a compromised query key — order matters, or the app goes down:

```
1  point the app at the SECONDARY admin key   app keeps running
2  regenerate the PRIMARY key                 compromised key dies
3  move the app back to the NEW primary key
```

Regenerating the key the app is currently using breaks it immediately.

Multimodal and Extraction Exam Gaps

Extend visual and extraction workflows with accessibility, safety, evidence, and cross-topic controls.

glasspaper oblicum

Visual Evidence and Accessibility

- **Alt-text:** concise, purposeful, and validated in context
- **Bounding regions:** preserve coordinates, confidence, and source references
- **Video:** retain timestamps, speakers, objects, and segment evidence
- **Safety:** screen visual content and protect against indirect prompt injection

Use a multimodal model for prose, and an extraction or detection workflow when automation needs evidence.

glasspaper oblicum

Extraction and Search Operations

- Use `prebuilt-layout` for document structure, tables, and Markdown
- Keep embedded figures with `AnalyzeOutputOption.FIGURES`
- Use object, table, or file projections for the required downstream shape
- Use hybrid search when lexical and semantic evidence both matter
- Filter document results by the user's allowed groups

Cross-topic controls: streaming audio, server VAD, `silence_duration_ms`, and `input_audio_buffer.speech_started` support low-latency voice; `Not(IsBlank(Local.Var1))` tests a workflow value.

Retrieval Practice — Lab 5

Pause here and complete Lab 5, then check its solution.

glasspaper oblicum

Key Takeaways

1. **Responses API** — role `"developer"`, type `"input_image"`; result `response.output_text`
2. **ChatCompletions API** — role `"system"`, type `"image_url"` (nested `{"url": ...}`); **Phi-4 only supports this**
3. **Image generation** — `client.images.generate(model=, prompt=, n=, size=)` → `data[0].b64_json`
4. **Sora video** — create → poll → download; `seconds` as string; 24-hour expiry; max 2 concurrent
5. **Content Understanding** — `ContentUnderstandingClient`, `begin_analyze()`, `AnalysisInput(data=bytes)`; field methods: extract/classify/generate; thresholds: 0.9/0.7/<0.7
6. **Document Intelligence** — `read`, `layout`, prebuilt, custom template/neural/composed/classifier; `DocumentAnalysisClient`; ≤500 MB

Moderating Uploaded Images

[EXAM]

Users upload photos to an agent. The requirement is to **classify harmful visual content and block by severity** — that is **image moderation** in Azure AI Content Safety.

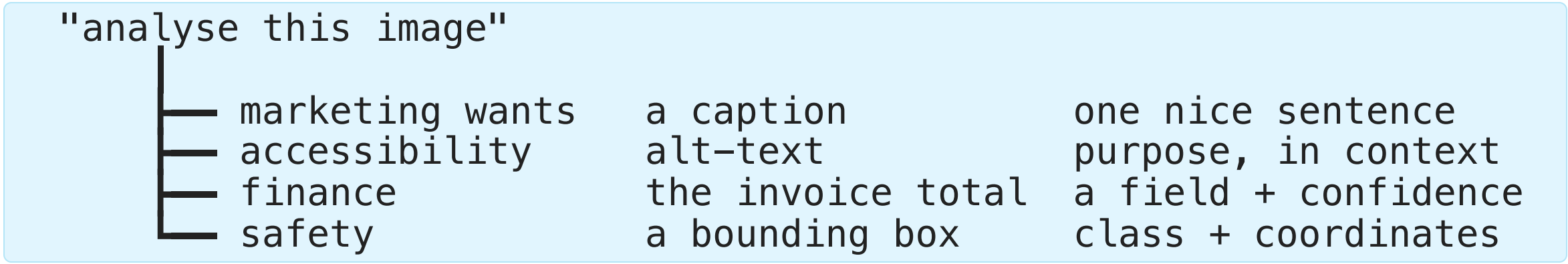
```
from azure.ai.contentsafety import ContentSafetyClient
from azure.ai.contentsafety.models import AnalyzeImageOptions, ImageData

request = AnalyzeImageOptions(image=ImageData(content=image_bytes))
response = client.analyze_image(request)   # severity per harm category
```

Control	What it inspects
Image moderation	The **picture itself** — harm category and severity
OCR keyword scanning	Only text extracted from the image
Prompt shields	Injected instructions, not visual harm
Blocklists	Configured phrases, reactive and text-only

Two different image risks: moderation stops a harmful *picture*; prompt shields stop *hidden instructions* inside it. A solution that accepts uploads usually needs both.

Problem: Everyone Says "Analyse the Image"

Four people say the same sentence and mean four different things.



Need: name the required *output* first. The service then follows almost automatically.

Picking a service before naming the output is the most common mistake in this area — on the exam and in practice.

Prose or Evidence?

The deepest split in this topic is not "which product", it is **what the answer is made of**.

```
PROSE                                   EVIDENCE
a model describes                       an extractor locates

"The invoice total is 1,240 EUR"        total: 1240.00
                                        confidence: 0.94
                                        page 2, polygon [x,y,...]

great for humans                        required for automation,
cannot be verified automatically        review queues, and audits
```

Ask: will a human read this, or will a system act on it? Only the second needs coordinates, confidence, and citations.

glasspaper oblicum

Visual Understanding Patterns

[EXAM]

A multimodal solution may need several distinct outputs:

Output	Example
Caption	“A delivery truck is parked beside a loading dock.”
Alt-text	A concise description usable by a screen reader
Visual question answering	“What color is the truck?”
Grounded extraction	Field value plus location or bounding region
Object detection	Object class plus coordinates and confidence

Choose the service by output:

- Multimodal model: flexible reasoning over images
- Content Understanding: configurable multimodal extraction
- Document Intelligence: document layout, tables, and fields
- Computer vision detection APIs: objects, tags, faces, or regions

Custom Vision — Train a Classifier

[EXAM]

Classifying manufactured components as faulty or acceptable:

1. **Create a project** — choose **classification**, not object detection
2. **Upload and tag images** — this **is** how the training dataset is created
3. **Train the model**

Distractor	Why it is wrong
“Initialize the training dataset”	Not a step — the dataset comes from uploading and tagging images
Train the **object detection** model	Object detection locates objects with bounding boxes; this assigns a label

Precision and Recall

[EXAM]

```
precision = TP / (TP + FP)     of what I predicted, how much was correct
recall    = TP / (TP + FN)     of what existed, how much did I find
```

In a training summary	Means
Precision 100%	No prediction was wrong → **0% false positives**
Recall 100%	Every actual instance was found

Trap: `TP / (TP + FP)` computes **precision**, so it is not the answer when the question asks for recall.

Retrieval Practice — Lab 6

Pause here and complete Lab 6 before continuing.

glasspaper oblicum

Accessibility and Alt-Text

[EXAM]

Alt-text is not the same as a marketing caption.

Good alt-text should:

- Convey the purpose or information of the image
- Be concise and specific
- Identify relevant text, people, objects, and relationships
- Avoid guessing sensitive attributes
- Avoid “image of” unless it improves clarity
- Respect the surrounding page context

Example:

```
Poor: An image.
Caption: A modern office with people working.
Alt-text: Three people review a dashboard beside a wall display showing monthly sales.
```

Video and Region-Level Analysis

[EXAM]

For video, process the right unit of content:

1. Identify the relevant time range
2. Sample frames or use a video analyzer
3. Detect speech, objects, scene changes, and actions
4. Preserve timestamps and confidence values
5. Summarize or extract fields with source references

For images, region-level results may include:

- Object label and confidence
- Bounding box or polygon
- Text location from OCR
- A field value grounded to a page or region

Exam distinction: a generative model can describe what it sees, but a detection or extraction workflow should return structured coordinates and evidence when the application needs them.

Visual Safety and Policy Controls

[EXAM]

Safety must cover both the prompt and the visual content.

Risk	Possible control
Unsafe generated image	Content filters, moderation, human review
Hidden instruction in an image	Indirect prompt-injection detection and prompt shields
Prohibited symbol or	Vision classifier, policy rules, review

Layer the controls: model choice → input screening → prompt protections → output screening → user disclosure → monitoring.

MSLearn status: prompt shields and responsible AI are covered; this specific visual-policy checklist is an exam-oriented synthesis.

Retrieval Practice — Lab 7

Pause here and complete Lab 7, then check its solution.

glasspaper oblicum

Document Intelligence vs. Content Understanding

[EXAM]

Requirement	Prefer
Read printed or handwritten text	Document Intelligence `read`
Tables, layout, selection marks, key-value pairs	Document Intelligence `layout`
Invoice, receipt, ID, or other prebuilt fields	Document Intelligence prebuilt model
Fixed or variable custom document forms	Document Intelligence custom model
Multimodal fields across documents, images, audio, and video	Content Understanding analyzer
Extract, classify, or generate custom fields	Content Understanding field schema

Confidence is not correctness: route uncertain fields to review and retain source grounding such as page, polygon, timestamp, or document reference.

Retrieval Practice — Lab 8

Pause here and complete Lab 8, then check its solution.

glasspaper oblicum

Azure AI Search and Knowledge Mining

[EXAM]

Knowledge-mining pipeline:

```
Data source → indexer → skillset → index → query or knowledge store
```

Exam decisions:

- Full-text/BM25: exact lexical matching
- Semantic ranking: language-aware reranking
- Vector search: semantic similarity using embeddings
- Hybrid search: combine lexical and vector evidence
- Knowledge store: persist enriched projections for analytics or export

A production index also needs:

- A document key and stable source metadata

Translation Beyond Text

[EXAM]

Scenario	Service or workflow
Text translation	Azure Translator `translate()`
Script conversion	Translator `transliterate()`
Speech	

Design choice: translating before retrieval may improve cross-language search, while translating after retrieval may preserve the original source for citations. Test both with the target dataset.

MSLearn status: text, transliteration, and speech translation are directly covered; document translation is less developed in the scraped course.

Cross-Topic Exam Controls

Some exam scenarios combine topics. For low-latency voice agents, streaming audio, server-side VAD, and `silence_duration_ms` reduce turn latency; `input_audio_buffer.speech_started` signals barge-in. In a workflow, `Not(IsBlank(Local.Var1))` tests that a scoped variable has a value.

Retrieval Practice — Lab 9

Pause here and complete Lab 9 before continuing.

glasspaper oblicum

Key Takeaways

1. **Output determines service:** caption, alt-text, detection, extraction, and video analysis are different tasks
2. **Ground visual answers:** preserve regions, pages, timestamps, confidence, and citations
3. **Accessibility is a quality requirement:** validate generated alt-text in context
4. **Visual safety needs layers:** filters, prompt shields, policy rules, disclosure, and review
5. **Choose extraction deliberately:** Document Intelligence for document structure; Content Understanding for multimodal analyzers
6. **Search operations matter:** monitor indexing, refreshes, failures, and relevance, not just query syntax
7. **Translation scope matters:** text, speech, scripts, documents,

Retrieval Practice — Lab 10

Pause here and complete Lab 10, then check its solution.

glasspaper oblicum

Appendix

Relationship to the Microsoft Learn path

glasspaper oblicum

Coverage Map

Exam objective	MSLearn status
Multimodal prompts and visual grounding	**Direct** — dedicated vision lesson
Image and video generation/editing	**Direct** — dedicated lessons
Accessibility descriptions and alt-text	**Indirect** — accessibility is mentioned, but not taught as a visual-output workflow
Video segments and object/region detection	**Partial** — Content Understanding is covered; detailed vision detection patterns are lighter
Visual policy controls and watermarking	**Exam gap** — not developed as a dedicated objective
Document Intelligence and Content Understanding	**Direct** — multiple detailed lessons
Search indexing, enrichment, and RAG	**Direct** — dedicated AI Search lesson
Document translation	**Partial** — text, speech translation, and transliteration are

Interpretation: this topic adds exam vocabulary and decision patterns around areas that the official lessons either mention briefly or treat through a different service.

Retrieval Practice — Topic 7

Pause after each matching section and answer the checkpoint questions before continuing:

- **Content Safety:** Questions 1–3
- **Multimodal input:** Questions 4–6
- **Document extraction:** Questions 7–9
- **Speech and workflow:** Questions 10–12
- **Search and privacy:** Questions 13–15

Open Topic 7, Lab 6 · Open the Lab 6 solution

glasspaper oblicum

Retrieval Practice — Topic 4

Pause after each matching section and answer the checkpoint questions before continuing:

- **Vision:** Questions 1–3
- **Generation:** Questions 4–6
- **Document Intelligence:** Questions 7–9
- **Content Understanding:** Questions 10–12
- **Safety and accessibility:** Questions 13–15

Open Topic 4, Lab 1 · Open the Lab 1 solution

glasspaper oblicum

Official Microsoft Slides

Open the official Topic 4 slides in original order

glasspaper oblicum